

CS543 WEB SERVICES

ASSIGNMENT 3

Integrate an External SOAP Partner

CampusEats — External Payment Gateway Integration

Team ID: 12

Team Members:

1. Mahi verma	20252651032
2. Subham kumar Mohanty	20252651057
3. Abhishek Jadhao	20252651003
4. Harsh N Shende	20252651023
5. Rishabh Mishra	20252651041

Table of Contents

1. Context and SOAP Integration Justification
2. Partner Service Contract
3. SOAP Request
4. SOAP Success Response
5. SOAP Fault
6. HTTP Binding
7. Service Discovery / Modern Catalogue
8. Fault Mapping
9. Complete Integration Flow
10. Design-Time Scope
11. Submission Artifacts

1. CONTEXT AND SOAP INTEGRATION JUSTIFICATION

CampusEats needs to integrate an external payment gateway to process payments. The selected operation is **charge**, where CampusEats sends payment and order information to the partner. The external partner returns either a successful response or a SOAP Fault. SOAP is used specifically at this external partner boundary, while the rest of CampusEats internal APIs remain REST-based. SOAP is justified here because the assignment requires a strong contract through WSDL, structured SOAP messages/faults, message-level security, and transaction-oriented integration needs.

CampusEats internal APIs → REST

External Payment Partner boundary → SOAP

2. PARTNER SERVICE CONTRACT

The `partner.wsdl` file is the formal service contract.

Field	Value
Service	CampusEatsPaymentPartnerService
Port Type	PaymentPortType
Operation	charge
Binding	PaymentSoapBinding
Style	document/literal
Transport	SOAP over HTTP
Endpoint	<code>https://payments.partner.example.com/soap/PaymentService</code>

WSDL structure:

- **types:** defines the data structures
- **message:** defines exchanged messages
- **portType:** defines the operation
- **binding:** defines how the operation is communicated
- **service/port:** defines the service endpoint

3. SOAP REQUEST

CampusEats sends a SOAP request when it needs to charge payment.

```
CampusEats
  ↓
SOAP Request
  ↓
External Payment Gateway
```

SOAP structure:

```
SOAP Envelope
|
|-- Header
|   |-- Credentials
|       |-- merchantId
|       |-- apiKey
|
|-- Body
    |-- chargeRequest
        |-- merchantId
        |-- orderId
        |-- amount
        |-- currency
        |-- paymentToken
```

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:pay="http://api.campuseats.example.com/payment">

  <soap:Header>
    <pay:Credentials>
      <pay:merchantId>CE-MERCHANT-001</pay:merchantId>
      <pay:apiKey>REDACTED-DEMO-KEY</pay:apiKey>
    </pay:Credentials>
  </soap:Header>

  <soap:Body>
    <pay:chargeRequest>
      <pay:merchantId>CE-MERCHANT-001</pay:merchantId>
      <pay:orderId>CE-ORDER-10045</pay:orderId>
      <pay:amount>499.00</pay:amount>
      <pay:currency>INR</pay:currency>
      <pay:paymentToken>tok_demo_7f83a1</pay:paymentToken>
    </pay:chargeRequest>
  </soap:Body>
</soap:Envelope>
```

Brief explanation:

- **Header:** contains credentials and additional processing information.
- **Body:** contains the actual charge operation and its data.

4. SOAP SUCCESS RESPONSE

When the payment is successful, the partner returns a SOAP response.

```
SOAP Envelope
|
|-- Body
|   |-- chargeResponse
|       |-- transactionId
|       |-- status
|       |-- message
```

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:pay="http://api.campuseats.example.com/payment">

  <soap:Body>
    <pay:chargeResponse>
      <pay:transactionId>TXN-20260824-10045</pay:transactionId>
      <pay:status>APPROVED</pay:status>
      <pay:message>Payment approved</pay:message>
    </pay:chargeResponse>
  </soap:Body>
</soap:Envelope>
```

The response confirms payment approval and provides a transaction ID that CampusEats can use to associate the payment with the order.

5. SOAP FAULT

A payment may fail and the partner can return a SOAP Fault.

```
SOAP Envelope
|
|-- Body
|   |-- SOAP Fault
|       |-- faultcode
|       |-- faultstring
|       |-- detail
|           |-- PaymentError
|               |-- code
|               |-- message
```

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:pay="http://api.campusseats.example.com/payment">

  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Client</faultcode>
      <faultstring>Payment declined</faultstring>
      <detail>
        <pay:PaymentError>
          <pay:code>card_declined</pay:code>
          <pay:message>
            The payment instrument was declined.
          </pay:message>
        </pay:PaymentError>
      </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

Brief explanation:

- **faultcode:** Standard code indicating where the fault occurred (e.g., Client).
- **faultstring:** Human-readable message (Payment declined).
- **detail:** Partner-specific error structure containing their internal code and exact message.

6. HTTP BINDING

SOAP is transported over HTTP using an HTTP POST request.

```
POST /soap/PaymentService HTTP/1.1
Host: payments.partner.example.com
Content-Type: text/xml; charset=utf-8
SOAPAction: "http://api.campusseats.example.com/payment/charge"
Content-Length: <calculated-length>
```

- **POST:** HTTP method used to send the SOAP message.
- **Host:** identifies the partner server.
- **Content-Type:** identifies the XML SOAP payload.
- **SOAPAction:** identifies the charge operation and comes from the WSDL binding.
- **Endpoint:** <https://payments.partner.example.com/soap/PaymentService>

```
WSDL
↓
SOAP binding
↓
SOAPAction + endpoint
↓
HTTP POST
↓
Payment Partner
```

7. SERVICE DISCOVERY / MODERN CATALOGUE

CampusEats does not run a live UDDI server for this assignment. Instead, the partner can be discovered through a modern service catalogue/registry or through a WSDL URL supplied during onboarding.

```
CampusEats
↓
Service Catalogue / Registry
↓
Find Payment Service
↓
Obtain WSDL + Endpoint
↓
Use WSDL Contract
↓
Send SOAP Request
```

Field	Registry Entry
Business	CampusEats Payment Gateway Partner
Service	CampusEatsPaymentPartnerService
Service Type	SOAP
Operation	charge
Endpoint	https://payments.partner.example.com/soap/PaymentService
WSDL	https://payments.partner.example.com/wsdl/PaymentService.wsdl
tModel-style pointer	urn:campuseats:payment:soap:PaymentPortType:v1

Distinction:

- **Registry:** helps discover/find the service.
- **WSDL:** describes the service contract and how to interact with it.

8. FAULT MAPPING

The external partner may use its own error vocabulary, such as: `card_declined`
CampusEats should not expose this partner-specific term directly to students.

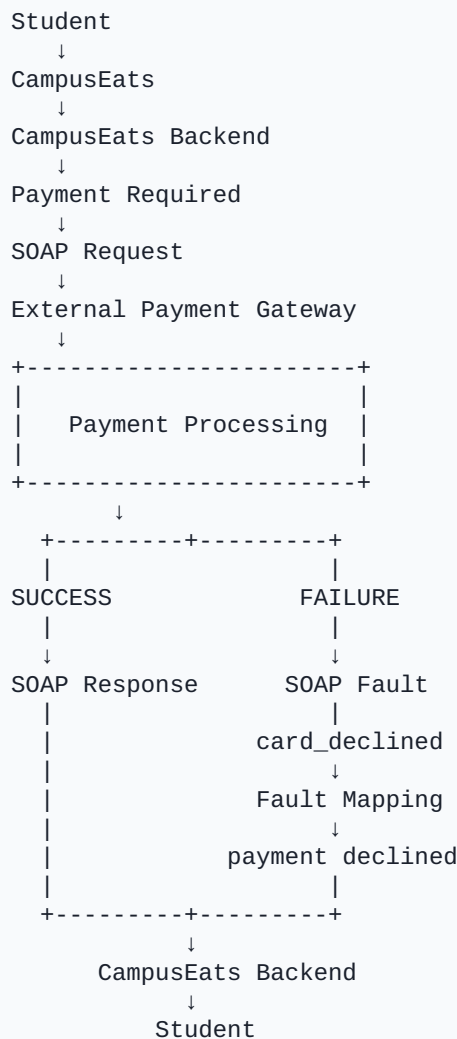
```
External Payment Partner
    |
    | card_declined
    ↓
CampusEats Integration Layer
    |
    | map external fault
    ↓
CampusEats Payments Service
    |
    | payment declined
    ↓
Student / CampusEats Client
```

- **Partner-specific error:** `card_declined`
- **Mapped CampusEats error:** `payment declined`

Fault mapping keeps the partner's internal implementation-specific vocabulary behind the integration boundary and allows CampusEats to maintain a consistent application-level error.

Assignment 2 Contract Mapping: Assignment 2's Payments Service defines `charge` with `orderId`, `amount`, and `paymentMethod` as inputs and `payment declined` as one of its errors. The integration effectively maps: `external card_declined` → `CampusEats Payments Service payment declined`.

9. COMPLETE INTEGRATION FLOW



The flow begins with the student ordering, which triggers the CampusEats Backend to send a SOAP Request to the External Payment Gateway. The payment is processed, resulting in either a SUCCESS (SOAP Response) or FAILURE (SOAP Fault mapping `card_declined` to `payment declined`). The mapped result is returned to the CampusEats Backend and finally to the student.

10. DESIGN-TIME SCOPE

This assignment is a design-time SOAP integration. No running SOAP server, live payment gateway, or live UDDI server is implemented. The WSDL, SOAP messages, HTTP binding, discovery entry, and fault mapping demonstrate how the integration would work conceptually.

11. SUBMISSION ARTIFACTS

File	Purpose
integration.pdf	Written integration report covering context, HTTP binding, discovery/registry, and fault mapping.
partner.wsdl	Partner SOAP service contract.
soap-request.xml	SOAP request for the charge operation.
soap-response.xml	Successful SOAP response.
soap-fault.xml	SOAP Fault for a declined payment.

Note: The `service-registry.json` and `http-request.txt` are supporting files used during the design process.